出处说明：本文是论文《TinyBERT: Distilling BERT for Natural Language Understanding》（Xiaoqi Jiao 等，2020）的中文译本，原文链接：https://arxiv.org/abs/1909.10351 （译自 arXiv v5, 2020 年 10 月 16 日版）

TinyBERT：为自然语言理解蒸馏 BERT

原题： TinyBERT: Distilling BERT for Natural Language Understanding

Xiaoqi Jiao[1]*†（焦晓奇）, Yichun Yin[2]*‡（尹伊淳）, Lifeng Shang[2]‡（尚利峰）, Xin Jiang[2]（蒋欣）, Xiao Chen[2]（陈晓）, Linlin Li[3]（李琳琳）, Fang Wang[1]‡（王芳）, Qun Liu[2]（刘群）

[1] 华中科技大学信息存储系统重点实验室，武汉光电国家研究中心（Key Laboratory of Information Storage System, Huazhong University of Science and Technology, Wuhan National Laboratory for Optoelectronics）[2] 华为诺亚方舟实验室（Huawei Noah’s Ark Lab）[3] 华为技术有限公司（Huawei Technologies Co., Ltd.）

{jiaoxiaoqi,wangfang}@hust.edu.cn {yinyichun,shang.lifeng,jiang.xin}@huawei.com {chen.xiao2,lynn.lilinlin,qun.li

* 作者贡献相同。† 本工作完成于焦晓奇在华为诺亚方舟实验室实习期间。‡ 通讯作者。

译者注：作者中文名为译者根据常见写法添加，原文只有英文名。文中「译者注」均为译者为初学者添加，原文没有。公式用 LaTeX 写出；参考文献保留英文原文。原文中 BERT_BASE、TinyBERT_4 等下标在下文写作 BERTBASE、TinyBERT4，下标数字表示层数。

摘要（Abstract）

以 BERT 为代表的语言模型预训练（language model pre-training）显著提升了许多自然语言处理任务的表现。然而，预训练语言模型的计算开销通常很大，因此很难在资源受限的设备上高效运行。为了在保持准确率的同时加速推理（inference）、减小模型规模，我们首先提出一种新颖的 Transformer 蒸馏方法，它是专门为基于 Transformer 的模型的知识蒸馏（knowledge distillation，KD）而设计的。利用这种新的 KD 方法，大型「教师」（teacher）BERT 中编码的大量知识可以有效地迁移到小型「学生」（student）TinyBERT 中。接着，我们为 TinyBERT 提出一个新的两阶段学习框架（two-stage learning framework），它在预训练阶段和特定任务学习阶段都执行 Transformer 蒸馏。这个框架保证 TinyBERT 既能捕获 BERT 中的通用领域（general-domain）知识，也能捕获特定任务（task-specific）知识。

4 层的 TinyBERT4[1] 在实验中很有效：在 GLUE 基准上取得了其教师 BERTBASE 96.8% 以上的性能，同时模型小 7.5 倍、推理快 9.4 倍。TinyBERT4 也明显优于 BERT 蒸馏方向上最先进的 4 层基线模型，而参数量只有它们的约 28%，推理时间只有它们的约 31%。此外，6 层的 TinyBERT6 与其教师 BERTBASE 表现相当。

[1] 代码和模型已公开：https://github.com/huawei-noah/Pretrained-Language-Model/tree/master/TinyBERT

---

1 引言（Introduction）

先预训练语言模型、再在下游任务上微调（fine-tuning），已经成为自然语言处理（NLP）的新范式。BERT（Devlin et al., 2019）、XLNet（Yang et al., 2019）、RoBERTa（Liu et al., 2019）、ALBERT（Lan et al., 2020）、

T5（Raffel et al., 2019）和 ELECTRA（Clark et al., 2020）等预训练语言模型（pre-trained language models, PLMs）在许多 NLP 任务上取得了巨大成功（例如 GLUE 基准（Wang et al., 2018）以及具有挑战性的多跳推理（multi-hop reasoning）任务（Ding et al., 2019））。然而，PLM 通常参数量巨大、推理耗时长，难以部署在手机等边缘设备上。最近的研究（Kovaleva et al., 2019; Michel et al., 2019; Voita et al., 2019）表明，PLM 中存在冗余。因此，在保留 PLM 性能的同时降低其计算开销和模型存储，既关键也可行。

人们已经提出了许多模型压缩技术（Han et al., 2016），用来在保持准确率的同时加速深度模型推理、减小模型规模。最常用的技术包括量化（quantization）（Gong et al., 2014）、权重剪枝（weights pruning）（Han et al., 2015）以及知识蒸馏（KD）（Romero et al., 2014）。本文聚焦于知识蒸馏，这一思想源自 Hinton et al.（2015），采用教师–学生框架（teacher-student framework）。KD 的目标是把大型教师网络中蕴含的知识迁移到小型学生网络中，训练学生网络去复现教师网络的行为。在这一框架基础上，我们提出一种专门针对基于 Transformer 的模型（Vaswani et al., 2017）的新蒸馏方法，并以 BERT 为例，研究该方法在大规模 PLM 上的应用。

KD 在 NLP 中已有大量研究（Kim and Rush, 2016; Hu et al., 2018），在预训练语言模型上也有研究（Sanh et al., 2019; Sun et al., 2019, 2020; Wang et al., 2020）。「先预训练再微调」范式先在大规模无监督文本语料上预训练 BERT，然后在特定任务数据集上微调，这大大增加了 BERT 蒸馏的难度。因此，需要为两个训练阶段都设计有效的 KD 策略。

> **图 1**（未重绘）：TinyBERT 学习过程示意图（The illustration of TinyBERT learning）。图中显示两条流程：大规模文本语料经「通用蒸馏」得到通用 TinyBERT；任务数据集经「数据增强」得到增强后的任务数据集，再经「特定任务蒸馏」得到微调后的 TinyBERT。

为了构建有竞争力的 TinyBERT，我们首先提出一种新的 Transformer 蒸馏方法，用来蒸馏教师 BERT 中蕴含的知识。具体来说，我们设计了三类损失函数，去拟合 BERT 各层的不同表示：1）嵌入层（embedding layer）的输出；2）由 Transformer 层得到的隐藏状态（hidden states）和注意力矩阵（attention matrices）；3）预测层（prediction layer）输出的 logits。基于注意力的拟合，受到了最近一项发现（Clark et al., 2019）的启发：BERT 学到的注意力权重能捕获大量语言学知识，因此这种拟合有助于把语言学知识从教师 BERT 很好地迁移到学生 TinyBERT。然后，我们提出一个新颖的两阶段学习框架，包括通用蒸馏（general distillation）和特定任务蒸馏（task-specific distillation），如图 1 所示。在通用蒸馏阶段，未经微调的原始 BERT 作为教师模型，学生 TinyBERT 在通用领域语料上，通过我们提出的 Transformer 蒸馏模仿教师的行为。之后，我们得到一个通用 TinyBERT，用作进一步蒸馏时学生模型的初始化。在特定任务蒸馏阶段，我们先做数据增强（data augmentation），然后以微调过的 BERT 作为教师模型，在增强后的数据集上进行蒸馏。需要指出的是，这两个阶段对于提升 TinyBERT 的性能和泛化能力都必不可少。

本工作的主要贡献如下：1）我们提出一种新的 Transformer 蒸馏方法，促使教师 BERT 中编码的语言学知识能被充分地迁移到 TinyBERT；2）我们提出一个新颖的两阶段学习框架，在预训练和微调阶段都执行所提出的 Transformer 蒸馏，从而保证 TinyBERT 能吸收教师 BERT 的通用领域知识和特定任务知识；3）实验表明，我们的 TinyBERT4 在 GLUE 任务上能达到教师 BERTBASE 96.8% 以上的性能，而参数少得多（约 13.3%）、推理时间短得多（约 10.6%），并且明显优于 BERT 蒸馏方向上其他最先进的 4 层基线；4）我们还表明，6 层的 TinyBERT6 在 GLUE 上能与教师 BERTBASE 表现相当。

---

2 预备知识（Preliminaries）

本节介绍 Transformer（Vaswani et al., 2017）和知识蒸馏（Hinton et al., 2015）的形式化描述。我们提出的 Transformer 蒸馏是一种专为基于 Transformer 的模型设计的 KD 方法。

2.1 Transformer 层（Transformer Layer）

最近的大多数预训练语言模型（例如 BERT、XLNet 和 RoBERTa）都由 Transformer 层构建，它们能通过自注意力机制（self-attention mechanism）捕获输入词元（token）之间的长距离依赖。具体来说，一个标准的 Transformer 层包含两个主要子层: 多头注意力（multi-head attention, MHA）和全连接前馈网络（feed-forward network，FFN）。

多头注意力（Multi-Head Attention，MHA）。注意力函数的计算依赖于三个组成部分：查询（queries）、键（keys）和值（values），分别记为矩阵 Q、K 和 V。注意力函数可以写成:

$$A = \frac{QK^T}{\sqrt{d_k}}, \tag{1}$$

$$\text{Attention}(Q, K, V) = \text{softmax}(A)\, V, \tag{2}$$

其中 d_k 是键的维度，作为缩放因子；A 是注意力矩阵，由 Q 和 K 通过点积运算计算相容度得到。函数的最终输出是值 V 的加权和，权重通过对矩阵 A 的每一列做 softmax() 运算得到。根据 Clark et al.（2019）的研究，BERT 中的注意力矩阵能捕获大量语言学知识，因此在我们提出的蒸馏方法中起着关键作用。

多头注意力通过拼接来自不同表示子空间（representation subspaces）的注意力头来定义:

$$\text{MHA}(Q, K, V) = \text{Concat}(h_1, \dots, h_k)\, W, \tag{3}$$

其中 k 是注意力头的数量，h_i 表示第 i 个注意力头，它由 Attention() 函数以来自不同表示子空间的输入计算得到。矩阵 W 充当一个线性变换。

逐位置前馈网络（Position-wise Feed-Forward Network，FFN）。Transformer 层还包含一个全连接前馈网络，形式如下:

$$\text{FFN}(x) = \max(0,\, xW_1 + b_1)\, W_2 + b_2. \tag{4}$$

可以看到，FFN 包含两个线性变换和一个 ReLU 激活。

2.2 知识蒸馏（Knowledge Distillation）

KD 的目标是把大型教师网络 T 的知识迁移到小型学生网络 S。学生网络被训练去模仿教师网络的行为。令 f^T 和 f^S 分别表示教师网络和学生网络的行为函数（behavior functions）。行为函数的作用是把网络输入转换成某种富含信息的表示，它可以定义为网络中任意一层的输出。在 Transformer 蒸馏的场景下，MHA 层或 FFN 层的输出，或者某些中间表示（例如注意力矩阵 A），都可以用作行为函数。形式上，KD 可以建模为最小化下面的目标函数:

$$\mathcal{L}_{KD} = \sum_{x \in \mathcal{X}} L(f^S(x),\, f^T(x)), \tag{5}$$

其中 $L(\cdot)$ 是衡量教师网络与学生网络之间差异的损失函数，x 是文本输入，$\mathcal{X}$ 表示训练数据集。因此，关键的研究问题就变成了如何定义有效的行为函数和损失函数。与以往的 KD 方法不同，除了特定任务的训练阶段之外，我们还需要考虑如何在 BERT 的预训练阶段进行 KD。

---

3 方法（Method）

本节我们为基于 Transformer 的模型提出一种新颖的蒸馏方法,并为从 BERT 蒸馏得到的模型(称为 TinyBERT）给出一个两阶段学习框架。

3.1 Transformer 蒸馏（Transformer Distillation）

所提出的 Transformer 蒸馏是专为 Transformer 网络设计的 KD 方法。在本工作中，学生网络和教师网络都由 Transformer 层构建。为了说明清楚，我们先对问题进行形式化，再介绍我们的方法。

> **图 2**（未重绘）：Transformer 层蒸馏的细节，包括 Attnloss（基于注意力的蒸馏）和 Hidnloss（基于隐藏状态的蒸馏）。子图 (a) 示意教师（BERT，N 层，隐藏维度 d）与学生（TinyBERT，M 层，隐藏维度 d'，$N > M$，$d > d'$）之间按层对应进行 Transformer 蒸馏，并包含嵌入层和预测层；子图 (b) 展示一对教师层和学生层内部（MHA、Add & Norm、FFN）在注意力矩阵（形状 $\mathbb{R}^{head\times l\times l}$）和隐藏状态（形状 $\mathbb{R}^{l\times d}$ 与 $\mathbb{R}^{l\times d'}$）上计算两种损失的位置。

问题形式化（Problem Formulation）。假设学生模型有 M 个 Transformer 层，教师模型有 N 个 Transformer 层。我们首先从教师模型的 N 层中选出 M 层，用于 Transformer 层蒸馏。然后定义一个函数 $n = g(m)$，作为从学生层索引到教师层索引的映射函数，表示学生模型的第 m 层从教师模型的第 $g(m)$ 层学习信息。准确地说，我们把嵌入层的索引设为 0，把预测层的索引设为 $M+1$，相应的层映射分别定义为 $0 = g(0)$ 和 $N+1 = g(M+1)$。不同映射函数的选择对性能的影响，会在实验部分讨论。形式上，学生通过最小化下面的目标，从教师那里获取知识：

$$\mathcal{L}_{model} = \sum_{x\in\mathcal{X}} \sum_{m=0}^{M+1} \lambda_m \, \mathcal{L}_{layer}\big(f_m^S(x),\, f_{g(m)}^T(x)\big), \tag{6}$$

其中 $\mathcal{L}_{layer}$ 指给定模型层（例如 Transformer 层或嵌入层）的损失函数，$f_m(x)$ 表示由第 m 层导出的行为函数，λ_m 是表示第 m 层蒸馏重要性的超参数（hyper-parameter）。

Transformer 层蒸馏（Transformer-layer Distillation）。所提出的 Transformer 层蒸馏包括基于注意力的蒸馏（attention based distillation）和基于隐藏状态的蒸馏（hidden states based distillation），如图 2 所示。基于注意力的蒸馏，动机来自最近的发现：BERT 学到的注意力权重能捕获丰富的语言学知识（Clark et al., 2019）。这类语言学知识包括句法（syntax）和共指（coreference）信息，它们对自然语言理解至关重要。因此我们提出基于注意力的蒸馏，促使语言学知识能从教师（BERT）迁移到学生（TinyBERT）。具体来说，学生学习拟合教师网络中多头注意力的矩阵，目标定义为：

$$\mathcal{L}_{attn} = \frac{1}{h}\sum_{i=1}^{h} \mathrm{MSE}\big(A_i^S,\, A_i^T\big), \tag{7}$$

其中 h 是注意力头的数量，$A_i \in \mathbb{R}^{l\times l}$ 指教师或学生第 i 个头对应的注意力矩阵，l 是输入文本长度，MSE() 表示均方误差（mean squared error）损失函数。在本工作中，我们用（未归一化的）注意力矩阵 A_i 作为拟合目标，而不是它的 softmax 输出 softmax(A_i)，因为我们的实验表明前者收敛更快、性能更好。

除了基于注意力的蒸馏，我们还从 Transformer 层的输出中蒸馏知识，目标如下：

$$\mathcal{L}_{hidn} = \text{MSE}(H^S W_h,\, H^T), \tag{8}$$

其中矩阵 $H^S \in \mathbb{R}^{l\times d'}$ 和 $H^T \in \mathbb{R}^{l\times d}$ 分别指学生网络和教师网络的隐藏状态，由式 (4) 计算得到。标量 d 和 d' 分别表示教师模型和学生模型的隐藏层大小（hidden size），为了得到更小的学生网络，d' 通常小于 d。矩阵 $W_h \in \mathbb{R}^{d'\times d}$ 是一个可学习的线性变换，它把学生网络的隐藏状态变换到与教师网络状态相同的空间。

> 译者注：pdftotext 提取的原文中，H^S 的形状显示为 $\mathbb{R}^{l\times d}$（撇号在提取时可能丢失）；按上下文，学生的隐藏维度为 d'，这里写作 $\mathbb{R}^{l\times d'}$。

嵌入层蒸馏（Embedding-layer Distillation）。与基于隐藏状态的蒸馏类似，我们也进行嵌入层蒸馏，目标为：

$$\mathcal{L}_{embd} = \text{MSE}(E^S W_e,\, E^T), \tag{9}$$

其中矩阵 E^S 和 E^T 分别指学生网络和教师网络的嵌入。在本文中，它们与隐藏状态矩阵的形状相同。矩阵 W_e 是一个线性变换，作用与 W_h 类似。

> 译者注：提取的原文此处写作「the matrices E^S and H^T」，应为 E^S 和 E^T 的笔误，译文按式 (9) 处理。

预测层蒸馏（Prediction-layer Distillation）。除了模仿中间层的行为，我们还像 Hinton et al.（2015）那样，用知识蒸馏去拟合教师模型的预测。具体来说，我们惩罚学生网络 logits 与教师 logits 之间的软交叉熵（soft cross-entropy）损失：

$$\mathcal{L}_{pred} = \text{CE}(z^T/t,\, z^S/t), \tag{10}$$

其中 z^S 和 z^T 分别是学生和教师预测的 logits 向量，CE 表示交叉熵（cross entropy）损失，t 表示温度（temperature）值。在我们的实验中，发现 $t = 1$ 效果很好。

利用上述蒸馏目标（即式 7、8、9 和 10），我们可以把教师网络与学生网络对应层之间的蒸馏损失统一起来：

$$\mathcal{L}_{layer} = \begin{cases} \mathcal{L}_{embd}, & m = 0 \\ \mathcal{L}_{hidn} + \mathcal{L}_{attn}, & M \geq m > 0 \\ \mathcal{L}_{pred}, & m = M+1 \end{cases} \tag{11}$$

3.2 TinyBERT 学习（TinyBERT Learning）

BERT 的应用通常包含两个学习阶段：预训练和微调。BERT 在预训练阶段学到的大量知识非常重要，应当迁移到压缩后的模型中。因此，我们提出一个新颖的两阶段学习框架，包括通用蒸馏和特定任务蒸馏，如图 1 所示。通用蒸馏帮助 TinyBERT 学习预训练 BERT 中蕴含的丰富知识，这对提升 TinyBERT 的泛化能力起着重

要作用。特定任务蒸馏则进一步把微调过的 BERT 中的知识教给 TinyBERT。通过这两步蒸馏，我们可以大幅缩小教师模型与学生模型之间的差距。

通用蒸馏（General Distillation）。我们以未经微调的原始 BERT 作为教师，以大规模文本语料作为训练数据。通过在通用领域文本上执行 Transformer 蒸馏 [2]，我们得到一个可以针对下游任务微调的通用 TinyBERT。然而，由于隐藏层/嵌入层维度和层数都大幅减小，通用 TinyBERT 的表现总体上不如 BERT。

[2] 在通用蒸馏中，我们不执行式 (10) 那样的预测层蒸馏。我们的动机是让 TinyBERT 在预训练阶段主要学习 BERT 的中间结构。根据初步实验，我们还发现，在已经执行了 Transformer 层蒸馏（Attn 和 Hidn 蒸馏）和嵌入层蒸馏的情况下，在预训练阶段再做预测层蒸馏，并不会给下游任务带来额外提升。

特定任务蒸馏（Task-specific Distillation）。以往研究表明，像微调后的 BERT 这样的复杂模型，在特定领域任务上存在过度参数化（over-parametrization）的问题（Kovaleva et al., 2019）。因此，更小的模型有可能取得与 BERT 相当的表现。为此，我们提出通过特定任务蒸馏来得到有竞争力的微调 TinyBERT。在特定任务蒸馏中，我们在增强后的特定任务数据集上，再次执行所提出的 Transformer 蒸馏。具体来说，以微调过的 BERT 作为教师，并提出一种数据增强方法来扩充特定任务的训练集。用更多与任务相关的样本训练，可以进一步提升学生模型的泛化能力。

算法 1 特定任务蒸馏的数据增强流程（Data Augmentation Procedure for Task-specific Distillation）

```
输入：x 是一个词序列
参数：p_t：阈值概率
      N_a：每个样本增强出的样本数
      K：候选集大小
输出：D'：增强后的数据
 1: n ← 0 ; D' ← [ ]
 2: while n < N_a do
 3:   x_m ← x
 4:   for i ← 1 to len(x) do
 5:     if x[i] 是单个词片（single-piece）的词 then
 6:       把 x_m[i] 替换为 [MASK]
 7:       C ← BERT(x_m)[i] 中概率最高的 K 个词
 8:     else
 9:       C ← 从 GloVe 中找出与 x[i] 最相似的 K 个词
10:     end if
11:     采样 p ~ Uniform(0, 1)
12:     if p ≤ p_t then
13:       用 C 中随机一个词替换 x_m[i]
14:     end if
15:   end for
16:   把 x_m 加入 D'
17:   n ← n + 1
18: end while
19: return D'
```

数据增强（Data Augmentation）。我们结合预训练语言模型 BERT 和 GloVe（Pennington et al., 2014）词向量（word embeddings），进行词级替换来实现数据增强。具体来说，对于单个词片的词（single-piece words），我们用语言模型预测替换词（Wu et al., 2019）；对于由多个词片组成的词（multiple-pieces words）[3]，我们用词向量检索最相似的词作为替换词。我们定义了一些超参数来控制一个句子中的替换比例以及增强数据集的规模。数据增强流程的更多细节见算法 1。在所有实验中，我们设 $p_t = 0.4$，$N_a = 20$，$K = 15$。

[3] 一个词会被 BERT 的分词器（tokenizer）切分成多个词片（word-pieces）。

上述两个学习阶段是互补的：通用蒸馏为特定任务蒸馏提供了良好的初始化，而在增强数据上的特定任务蒸馏则通过专注于学习特定任务知识，进一步改进 TinyBERT。尽管模型规模大幅缩小，借助数据增强，并在预训练和微调两个阶段都执行所提出的 Transformer 蒸馏方法，TinyBERT 能在多种 NLP 任务上取得有竞争力的表现。

---

4 实验（Experiments）

本节我们在多种任务、不同模型设置下，评估 TinyBERT 的有效性和效率。

4.1 数据集（Datasets）

我们在通用语言理解评测（General Language Understanding Evaluation，GLUE）（Wang et al., 2018）基准上评估 TinyBERT。它包含 2 个单句任务：CoLA（Warstadt et al., 2019）、SST-2（Socher et al., 2013）；3 个句子相似度任务：MRPC（Dolan and Brockett, 2005）、STS-B（Cer et al., 2017）、QQP（Chen et al., 2018）；以及 4 个自然语言推理（natural language inference）任务：MNLI（Williams et al., 2018）、QNLI（Rajpurkar et al., 2016）、RTE（Bentivogli et al., 2009）和 WNLI（Levesque et al., 2012）。这些任务的评价指标可在 GLUE 论文（Wang et al., 2018）中找到。

4.2 TinyBERT 设置（TinyBERT Settings）

我们实例化了一个微型学生模型（层数 $M = 4$，隐藏层大小 $d' = 312$，前馈/滤波器大小 $d'_i = 1200$，注意力头数 $h = 12$），总参数量为 14.5M，记为 TinyBERT4。原始 BERTBASE（$N = 12$，$d = 768$，$d_i = 3072$，$h = 12$）用作教师模型，包含 109M 参数。我们使用 $g(m) = 3 \times m$ 作为层映射函数，因此 TinyBERT4 每隔 3 层从 BERTBASE 学习一次。每一层的学习权重 λ 都设为 1。此外，为了与基线直接比较，我们还实例化了一个 TinyBERT6（$M = 6$，$d' = 768$，$d'_i = 3072$，$h = 12$），其架构与 BERT6-PKD（Sun et al., 2019）和 DistilBERT6（Sanh et al., 2019）相同。

TinyBERT 的学习包括通用蒸馏和特定任务蒸馏。在通用蒸馏中，我们把最大序列长度设为 128，使用英文维基百科（25 亿词）作为文本语料，在预训练 BERTBASE 的监督下进行 3 个 epoch 的中间层蒸馏，其余超参数与 BERT 预训练（Devlin et al., 2019）保持一致。在特定任务蒸馏中，在微调过的 BERT 监督下，我们先在增强数据上进行 20 个 epoch[4] 的中间层蒸馏，批大小为 32，学习率为 5e-5；然后在增强数据 [5] 上进行 3 个 epoch 的预测层蒸馏，批大小从 {16, 32} 中选取、学习率从 {1e-5, 2e-5, 3e-5} 中选取，依据开发集表现确定。在特定任务蒸馏中，单句任务的最大序列长度设为 64，句对任务设为 128。

[4] 对于大数据集 MNLI、QQP 和 QNLI，我们只进行 10 个 epoch 的中间层蒸馏；对于有挑战性的 CoLA 任务，这一步进行 50 个 epoch。[5] 对于回归任务 STS-B，使用原始训练集效果更好。

4.3 基线（Baselines）

我们把 TinyBERT 与 BERTTINY、BERTSMALL[6]（Turc et al., 2019），以及若干最先进的 KD 基线进行比较，包括 BERT-PKD（Sun et al., 2019）、PD（Turc et al., 2019）、DistilBERT（Sanh et al., 2019）和 MobileBERT（Sun et al., 2020）。BERTTINY 指直接预训练一个小 BERT，其模型架构与 TinyBERT4 相同。

训练 BERTTINY 时，我们遵循原始 BERT（Devlin et al., 2019）中描述的学习策略。为了公平比较，我们使用公开代码训练了一个 4 层的 BERT4-PKD[7] 和一个 4 层的 DistilBERT4[8]，并用推荐的超参数微调这些 4 层基线。对于 6 层基线，我们使用其论文报告的数字，或用公开模型在 GLUE 测试集上评估结果。

[6] https://github.com/google-research/bert [7] https://github.com/intersun/PKD-for-BERT-Model-Compression
[8] https://github.com/huggingface/transformers/tree/master/examples/distillation

4.4 GLUE 上的实验结果（Experimental Results on GLUE）

我们把模型预测提交到官方 GLUE 评测服务器，获得测试集结果 [9]，汇总在表 1 中。

[9] https://gluebenchmark.com

表 1： 结果在 GLUE 官方基准的测试集上评估。每组学生模型中的最佳结果在原文中以粗体标出（本译本未保留粗体）。TinyBERT4 和 BERTTINY 的架构为（$M = 4$, $d = 312$, $d_i = 1200$), BERTSMALL 为（$M = 4$, $d = 512$, $d_i = 2048$），BERT4-PKD 和 DistilBERT4 为（$M = 4$, $d = 768$, $d_i = 3072$），BERT6-PKD、DistilBERT6 和 TinyBERT6 为（$M = 6$, $d = 768$, $d_i = 3072$）。所有模型都以单任务方式学习。推理加速在单块 NVIDIA K80 GPU 上评估。† 表示 MobileBERTTINY 与 TinyBERT4 之间的比较可能不公平，因为前者有 24 层，并且是从 IB-BERTLARGE 进行与任务无关（task-agnostic）的蒸馏得到的，而后者是从 BERTBASE 进行特定任务蒸馏得到的 4 层模型。

系统	参数量	FLOPs	加速比	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	平均
BERTBASE（教师）	109M	22.5B	1.0x	83.9/83.4	71.1	90.9	93.4	52.8	85.2	87.5	67.0	79.5
BERTTINY	14.5M	1.2B	9.4x	75.4/74.9	66.5	84.8	87.6	19.5	77.1	83.2	62.6	70.2
BERTSMALL	29.2M	3.4B	5.7x	77.6/77.0	68.1	86.4	89.7	27.8	77.0	83.4	61.8	72.1
BERT4-PKD	52.2M	7.6B	3.0x	79.9/79.3	70.2	85.1	89.4	24.8	79.8	82.6	62.3	72.6
DistilBERT4	52.2M	7.6B	3.0x	78.9/78.0	68.5	85.2	91.4	32.8	76.1	82.4	54.1	71.9
MobileBERTTINY	15.1M	3.1B	-	81.5/81.6	68.9	89.5	91.7	46.7	80.1	87.9	65.1	77.0
TinyBERT4（本文）	14.5M	1.2B	9.4x	82.5/81.8	71.3	87.7	92.6	44.1	80.4	86.4	66.6	77.0
BERT6-PKD	67.0M	11.3B	2.0x	81.5/81.0	70.7	89.0	92.0	-	-	85.0	65.5	-
PD	67.0M	11.3B	2.0x	82.8/82.2	70.4	88.9	91.8	-	-	86.8	65.3	-
DistilBERT6	67.0M	11.3B	2.0x	82.6/81.3	70.1	88.9	92.5	49.0	81.3	86.9	58.4	76.8
TinyBERT6（本文）	67.0M	11.3B	2.0x	84.6/83.2	71.6	90.4	93.1	51.1	83.7	87.3	70.0	79.4

4 层学生模型的实验结果表明：1）由于模型规模大幅缩小，BERTTINY（或 BERTSMALL）与 BERTBASE 之间存在很大的性能差距。2）TinyBERT4 在所有 GLUE 任务上都一致优于 BERTTINY，平均提升高达 6.8%。这表明所提出的 KD 学习框架能有效提升小模型在多种下游任务上的表现。3）TinyBERT4 明显优于最先进的 4 层 KD 基线（即 BERT4-PKD 和 DistilBERT4），领先至少 4.4%，而参数量只有约 28%，推理加速 3.1 倍。4）与教师 BERTBASE 相比，TinyBERT4 在模型效率上小 7.5 倍、快 9.4 倍，同时保持了有竞争力的表现。5）在有挑战性的 CoLA 数据集（预测语言可接受性判断的任务）上，所有 4 层蒸馏模型与教师模型相比都有较大的性能差距，而 TinyBERT4 相对 4 层基线取得了显著提升。6）我们还把 TinyBERT 与 24 层的 MobileBERTTINY 进行了比较，后者是从 24 层的 IB-BERTLARGE 蒸馏得到的。结果表明，TinyBERT4 只用 38.7% 的 FLOPs 就达到了与这个 24 层模型相同的平均分。7）当我们把模型容量增加到 TinyBERT6 时，其性能可以进一步提升，平均比相同架构的基线高 2.6%，并取得与教师相当的结果。8）另一个两阶段基线 PD 先预训练一个小 BERT，再用这个小模型在特定任务上做蒸馏；与之相比，TinyBERT 在特定任务阶段通过通用蒸馏来初始化学生。我们在附录 C 中分析了这两种初始化方法。

此外，BERT-PKD 和 DistilBERT 用预训练 BERT 的部分层来初始化学生模型，这使得学生模型的 Transformer 层（或嵌入层）必须与教师保持相同的尺寸设置。在我们的两阶段蒸馏框架中，TinyBERT 通过通用蒸馏初始化，因此在选择模型配置上更加灵活。

更多比较（More Comparisons）。我们纳入了更多基线来证明 TinyBERT 的有效性，例如 Poor Man’s BERT（Sajjad et al., 2020）、BERT-of-Theseus（Xu et al., 2020）和 MiniLM（Wang et al., 2020），其中一些只报告了 GLUE 开发集上的结果。此外，我们还在 SQuAD v1.1 和 v2.0 上评估了 TinyBERT。由于篇幅所限，我们把这些结果放在附录 A 和 B 中。

4.5 消融研究（Ablation Studies）

本节我们进行消融研究，考察以下两方面的贡献：a）图 1 中所提出的两阶段 TinyBERT 学习框架的不同步骤；b）式 (11) 中不同的蒸馏目标。

表 2：两阶段学习框架中不同步骤（即 TD、GD 和 DA）的消融研究。各变体在开发集上验证。

系统	MNLI-m	MNLI-mm	MRPC	CoLA	平均
TinyBERT4	82.8	82.9	85.8	50.8	75.6
去掉 GD（w/o GD）	82.5	82.6	84.1	40.8	72.5
去掉 TD（w/o TD）	80.6	81.2	83.8	28.5	68.5
去掉 DA（w/o DA）	80.5	81.0	82.4	29.8	68.4

表 3： TinyBERT 学习中不同蒸馏目标的消融研究。各变体在开发集上验证。

系统	MNLI-m	MNLI-mm	MRPC	CoLA	平均
TinyBERT4	82.8	82.9	85.8	50.8	75.6
去掉 Embd（w/o Embd）	82.3	82.3	85.0	46.7	74.1
去掉 Pred（w/o Pred）	80.5	81.0	84.3	48.2	73.5
去掉 Trm（w/o Trm）	71.7	72.3	70.1	11.2	56.3
去掉 Attn（w/o Attn）	79.9	80.7	82.3	41.1	71.0
去掉 Hidn（w/o Hidn）	81.7	82.1	84.1	43.7	72.9

4.5.1 学习步骤的影响（Effects of Learning Procedure） 所提出的两阶段 TinyBERT 学习框架包括三个关键步骤：GD（通用蒸馏，General Distillation）、TD（特定任务蒸馏，Task-specific Distillation）和 DA（数

据增强，Data Augmentation）。分别去掉每个学习步骤后的表现分析见表 2。结果表明，这三个步骤对所提出的方法都至关重要。TD 和 DA 在全部四个任务上的作用相当。我们注意到，在所有任务上，特定任务的步骤（TD 和 DA）都比预训练步骤（GD）更有帮助。另一个有趣的观察是，GD 在 CoLA 上的贡献比在 MNLI 和 MRPC 上更大。我们推测，GD 学到的语言泛化能力（Warstadt et al., 2019）在语言可接受性判断任务中起着重要作用。

4.5.2 蒸馏目标的影响（Effects of Distillation Objective） 我们考察蒸馏目标对 TinyBERT 学习的影响。我们设置了几个基线，分别是不做 Transformer 层蒸馏（w/o Trm）、不做嵌入层蒸馏（w/o Emb）或不做预测层蒸馏（w/o Pred）[10] 的学习。结果如表 3 所示，说明所提出的所有蒸馏目标都是有用的。不做 Trm[11] 时，性能从 75.6 大幅下降到 56.3。大幅下降的原因在于学生模型的初始化。在预训练阶段，获得良好的初始化对基于 Transformer 的模型的蒸馏至关重要，而在 w/o Trm 设置下，这一阶段没有来自上层的监督信号去更新 Transformer 层的参数。此外，我们还研究了 Transformer 层蒸馏中注意力（Attn）和隐藏状态（Hidn）各自的贡献。可以发现，基于注意力的蒸馏比基于隐藏状态的蒸馏影响更大。同时，这两种知识蒸馏是互补的，这使它们成为我们实验中基于 Transformer 的模型最重要的蒸馏技术。

[10] 预测层蒸馏是在增强数据上执行式 (10) 那样的软交叉熵。「w/o Pred」指对原始训练集的真实标签（ground-truth）执行标准交叉熵。[11] 在「w/o Trm」设置下，我们实际上：1）在预训练阶段进行嵌入层蒸馏；2）在微调阶段进行嵌入层和预测层蒸馏。

4.6 映射函数的影响（Effects of Mapping Function）

我们还考察了不同映射函数 $n = g(m)$ 对 TinyBERT 学习的影响。第 4.2 节所述的原始 TinyBERT 使用均匀策略（uniform strategy），我们把它与两种典型基线进行比较：顶层策略（top-strategy，$g(m) = m+N-M;\ 0 < m \leq M$）和底层策略（bottom-strategy，$g(m) = m;\ 0 < m \leq M$）。

表 4： TinyBERT4 使用不同映射策略的结果（开发集）。

系统	MNLI-m	MNLI-mm	MRPC	CoLA	平均
均匀（Uniform）	82.8	82.9	85.8	50.8	75.6
顶层（Top）	81.7	82.3	83.6	35.9	70.9
底层（Bottom）	80.6	81.3	84.6	38.5	71.3

比较结果见表 4。我们发现，顶层策略在 MNLI 上比底层策略好，而在 MRPC 和 CoLA 上更差，这印证了「不同任务依赖于 BERT 不同层的知识」这一观察。均匀策略覆盖了 BERTBASE 从底层到顶层的知识，在所有任务上都比另外两个基线表现更好。为特定任务自适应地选择层是一个有挑战性的问题，我们留作未来工作。

> 译者注：均匀策略即正文用的 $g(m) = 3m$：学生第 1、2、3、4 层分别学习教师第 3、6、9、12 层。顶层策略学习教师最后 4 层（9–12 层），底层策略学习教师最前 4 层（1–4 层）。

5 相关工作（Related Work）

预训练语言模型压缩（Pre-trained Language Models Compression） 一般来说，预训练语言模型（PLM）可以通过以下方式压缩：低秩近似（low-rank approximation）（Ma et al., 2019; Lan et al., 2020）、权重共享（weight sharing）（Dehghani et al., 2019; Lan et al., 2020）、知识蒸馏（Tang et al., 2019; Sanh et al., 2019;

Turc et al., 2019; Sun et al., 2020; Liu et al., 2020; Wang et al., 2020）、剪枝（pruning）（Cui et al., 2019; McCarley, 2019; F. et al., 2020; Elbayad et al., 2020; Gordon et al., 2020; Hou et al., 2020）或量化（Shen et al., 2019; Zafrir et al., 2019）。本文聚焦于知识蒸馏。

PLM 的知识蒸馏（Knowledge Distillation for PLMs）已有一些工作尝试把预训练语言模型蒸馏成更小的模型。BiLSTMSOFT（Tang et al., 2019）把 BERT 中的特定任务知识蒸馏到单层 BiLSTM 中。BERT-PKD（Sun et al., 2019）在微调阶段不仅从教师的最后一层提取知识，还从中间层提取知识。DistilBERT（Sanh et al., 2019）在大规模语料上于预训练阶段进行蒸馏。同期工作中，MobileBERT（Sun et al., 2020）在预训练阶段通过渐进式知识迁移（progressive knowledge transfer），把一个加入了瓶颈结构（bottleneck structures）的 BERTLARGE 蒸馏成一个 24 层的瘦身版本。MiniLM（Wang et al., 2020）同样在预训练阶段进行深度自注意力蒸馏（deep self-attention distillation）。与之不同，我们提出一个新的两阶段学习框架，用一种新颖的 Transformer 蒸馏方法，在预训练和微调两个阶段都从 BERT 中蒸馏知识。

预训练轻量 PLM（Pretraining Lite PLMs）其他相关工作旨在直接预训练轻量的 PLM。Turc et al.（2019）预训练了 24 个微型 BERT 模型，表明在较小架构的情况下预训练仍然重要，并且微调预训练过的紧凑模型可以很有竞争力。ALBERT（Lan et al., 2020）结合嵌入分解（embedding factorization）和跨层参数共享（cross-layer parameter sharing）来减少模型参数。由于 ALBERT 并没有减小 Transformer 块的隐藏层大小或层数，它的计算量仍然很大。另一项同期工作 ELECTRA（Clark et al., 2020）提出了一种样本高效的任务，称为替换词元检测（replaced token detection），以加速预训练；它还给出了一个 12 层的 ELECTRAsmall，其表现与 TinyBERT4 相当。与这些小型 PLM 不同，TinyBERT4 是一个 4 层模型，可以实现更大的加速。

---

6 结论与未来工作（Conclusion and Future Work）

本文介绍了一种基于 Transformer 的蒸馏新方法，并进一步为 TinyBERT 提出了一个两阶段框架。大量实验表明，TinyBERT 在取得有竞争力的表现的同时，显著减小了 BERTBASE 的模型规模和推理时间，为在边缘设备上部署基于 BERT 的 NLP 模型提供了一种有效途径。在未来工作中，我们将研究如何把更宽、更深的教师（例如 BERTLARGE）中的知识有效地迁移到学生 TinyBERT 中。把蒸馏与量化/剪枝结合，是进一步压缩预训练语言模型的另一个有前景的方向。

致谢（Acknowledgements）

本工作部分受到国家自然科学基金（NSFC）No.61832020、No.61821003、61772216，国家科技重大专项 No.2017ZX01032-101，以及中央高校基本科研业务费（Fundamental Research Funds for the Central Universities）的资助。

参考文献（References）

L. Bentivogli, P. Clark, I. Dagan, and D. Giampiccolo. 2009. The fifth pascal recognizing textual entailment challenge.

D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia. 2017. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In Proceedings of the 11th International Workshop on Semantic Evaluation.

Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. Quora question pairs.

K. Clark, U. Khandelwal, O. Levy, and C. D. Manning. 2019. What does bert look at? an analysis of bert' s attention. In Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP.

K. Clark, M. Luong, Q. V. Le, and C. D. Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. In ICLR.

B. Cui, Y. Li, M. Chen, and Z. Zhang. 2019. Finetune bert with sparse self-attention mechanism. In EMNLP.

M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. 2019. Universal transformers. In ICLR.

J. Devlin, M. Chang, K. Lee, and K. Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In NAACL.

M. Ding, C. Zhou, Q. Chen, H. Yang, and J. Tang. 2019. Cognitive graph for multi-hop reading comprehension at scale. In ACL.

W. B. Dolan and C. Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In Proceedings of the Third International Workshop on Paraphrasing.

M. Elbayad, J. Gu, E. Grave, and M. Auli. 2020. Depth-adaptive transformer. In ICLR.

Angela F., Edouard G., and Armand J. 2020. Reducing transformer depth on demand with structured dropout. In ICLR.

Y. Gong, L. Liu, M. Yang, and L. Bourdev. 2014. Compressing deep convolutional networks using vector quantization. arXiv preprint arXiv:1412.6115.

M. A. Gordon, K. Duh, and N. Andrews. 2020. Compressing bert: Studying the effects of weight pruning on transfer learning. arXiv preprint arXiv:2002.08307.

S. Han, Mao H., and Dally W. J. 2016. Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding. In ICLR.

S Han, J. Pool, J. Tran, and W. Dally. 2015. Learning both weights and connections for efficient neural network. In NIPS.

G. Hinton, O. Vinyals, and J. Dean. 2015. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.

L. Hou, L. Shang, X. Jiang, and Q. Liu. 2020. Dynabert: Dynamic bert with adaptive width and depth. arXiv preprint arXiv:2004.04037.

M. Hu, Y. Peng, F. Wei, Z. Huang, D. Li, N. Yang, and M. Zhou. 2018. Attention-guided answer distillation for machine reading comprehension. In EMNLP.

Y. Kim and A. M. Rush. 2016. Sequence-level knowledge distillation. In EMNLP.

O. Kovaleva, A. Romanov, A. Rogers, and A. Rumshisky. 2019. Revealing the dark secrets of bert. In EMNLP.

Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. 2020. Albert: A lite bert for self-supervised learning of language representations. In ICLR.

Hector Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. In Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning.

W. Liu, P. Zhou, Z. Zhao, Z. Wang, H. Deng, and Q. Ju. 2020. Fastbert: a self-distilling bert with adaptive inference time. arXiv preprint arXiv:2004.02178.

Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692.

X. Ma, P. Zhang, S. Zhang, N. Duan, Y. Hou, M. Zhou, and D. Song. 2019. A tensorized transformer for language modeling. In NIPS.

J. S. McCarley. 2019. Pruning a bert-based question answering model. arXiv preprint arXiv:1910.06360.

P. Michel, O. Levy, and G. Neubig. 2019. Are sixteen heads really better than one? In NIPS.

J. Pennington, R. Socher, and C. D. Manning. 2014. Glove: Global vectors for word representation. In EMNLP.

C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. arXiv preprint arXiv:1910.10683.

P. Rajpurkar, R. Jia, and P. Liang. 2018. Know what you don’t know: Unanswerable questions for squad. In ACL.

P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In EMNLP.

A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. 2014. Fitnets: Hints for thin deep nets. arXiv preprint arXiv:1412.6550.

H. Sajjad, F. Dalvi, N. Durrani, and P. Nakov. 2020. Poor man’s bert: Smaller and faster transformer models. arXiv preprint arXiv:2004.03844.

V. Sanh, L. Debut, J. Chaumond, and T. Wolf. 2019. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. 2019. Q-bert: Hessian based ultra low precision quantization of bert. arXiv preprint arXiv:1909.05840.

R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In EMNLP.

S. Sun, Y. Cheng, Z. Gan, and J. Liu. 2019. Patient knowledge distillation for bert model compression. In EMNLP.

Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou. 2020. Mobilebert: a compact task-agnostic bert for resource-limited devices. arXiv preprint arXiv:2004.02984.

R. Tang, Y. Lu, L. Liu, L. Mou, O. Vechtomova, and J. Lin. 2019. Distilling task-specific knowledge from bert into simple neural networks. arXiv preprint arXiv:1903.12136.

I. Turc, M. Chang, K. Lee, and K. Toutanova. 2019. Well-read students learn better: The impact of student initialization on knowledge distillation. arXiv preprint arXiv:1908.08962.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. 2017. Attention is all you need. In NIPS.

E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In ACL.

A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP.

W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. arXiv preprint arXiv:2002.10957.

A. Warstadt, A. Singh, and S. R. Bowman. 2019. Neural network acceptability judgments. TACL.

A. Williams, N. Nangia, and S. Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In NAACL.

X. Wu, S. Lv, L. Zang, J. Han, and S. Hu. 2019. Conditional bert contextual augmentation. In International Conference on Computational Science.

C. Xu, W. Zhou, T. Ge, F. Wei, and M. Zhou. 2020. Bert-of-theseus: Compressing bert by progressive module replacing. arXiv preprint arXiv:2002.02925.

Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. In NIPS.

O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat. 2019. Q8bert: Quantized 8bit bert. arXiv preprint arXiv:1910.06188.

附录（Appendix）

A 在 GLUE 上的更多比较（More Comparisons on GLUE）

由于以往一些 BERT 压缩工作只在 GLUE 开发集上评估模型，为了方便直接比较，这里我们把 TinyBERT6 与这些工作报告的结果进行比较。所有参与比较的方法都与 TinyBERT6 具有相同的模型架构（即 $M = 6$, $d' = 768$, $d'_i = 3072$）。

表 5： TinyBERT 与其他基线在 GLUE 任务开发集上的比较。Mcc 指马修斯相关系数（Matthews correlation），Pear/Spea 指皮尔逊/斯皮尔曼相关系数（Pearson/Spearman）。括号内为各数据集训练集规模。

相同学生架构（$M = 6$; $d' = 768$; $d'_i = 3072$）：

系统	CoLA (8.5k) Mcc	MNLI-m (393k) Acc	MNLI-mm (393k) Acc	MRPC (3.7k) F1/Acc	QNLI (105k) Acc	QQP (364k) F1/Acc	RTE (2.5k) Acc	SST-2 (67k) Acc	STS-B (5.7k) Pear/Spea
DistilBERT6	51.3	82.2	-	87.5/-	89.2	-/88.5	59.9	92.7	-/86.9
Poor Man’s BERT6	-	81.1	-	-/80.2	87.6	-/90.4	65.0	90.3	-/88.5
BERT-of-Theseus	51.1	82.3	-	89.0/-	89.5	-/89.6	68.2	91.5	-/88.7
MiniLM6	49.2	84.0	-	88.4/-	91.0	-/91.0	71.5	92.0	-
TinyBERT6	54.0	84.5	84.5	90.6/86.3	91.1	88.0/91.1	73.4	93.0	90.1/89.6

译者注：表 5 的单元格对应关系依据 `pdftotext -layout` 的版面对齐整理。

直接比较的结果见表 5。可以看到，在相同的架构设置和评估方法下，TinyBERT6 优于所有基线。TinyBERT 的有效性得到了进一步证实。

B 在 SQuAD v1.1 和 v2.0 上的结果（Results on SQuAD v1.1 and v2.0）

我们还在问答（question answering, QA）任务 SQuAD v1.1（Rajpurkar et al., 2016）和 SQuAD v2.0（Rajpurkar et al., 2018）上证明了 TinyBERT 的有效性。按照以往工作（Devlin et al., 2019）中的学习流程，我们把这两个任务视为序列标注（sequence labeling）问题，预测每个词元作为答案片段（answer span）起点或终点的可能性。与 GLUE 任务的一个小区别是，我们在原始训练集而不是增强数据集上执行预测层蒸馏，这样可以带来更好的表现。

表 6: 基线和 TinyBERT 在问答任务上的结果（开发集）。MiniLM4 的架构为（$M = 4$, $d = 384$, $d_i = 1536$），比 TinyBERT4 更宽；MiniLM6 的架构与 TinyBERT6 相同（$M = 6$，$d = 768$，$d_i = 3072$）。

系统	SQuAD 1.1 EM	SQuAD 1.1 F1	SQuAD 2.0 EM	SQuAD 2.0 F1
BERTBASE（教师）	80.7	88.4	74.5	77.7
4 层学生模型				
BERT4-PKD	70.1	79.5	60.8	64.6
DistilBERT4	71.8	81.2	60.6	64.1
MiniLM4	-	-	-	69.7
TinyBERT4	72.7	82.1	68.2	71.8
6 层学生模型				
BERT6-PKD	77.1	85.3	66.3	69.8
DistilBERT6	78.1	86.2	66.0	69.5
MiniLM6	-	-	-	76.4
TinyBERT6	79.7	87.5	74.7	77.7

结果表明，TinyBERT 一致优于 4 层和 6 层基线，说明所提出的框架同样适用于词元级标注任务。与序列级的 GLUE 任务相比，问答任务需要依赖更细微的知识来推断正确答案，这增加了知识蒸馏的难度。如何构建更好的问答 TinyBERT（QA-TinyBERT），我们留作未来工作。

C 用 BERTTINY 初始化 TinyBERT（Initializing TinyBERT with BERTTINY）

在所提出的两阶段学习框架中, 为了让 TinyBERT 在不同下游任务上都能有效工作, 我们提出用通用蒸馏（GD）来捕获通用领域知识, TinyBERT 借此在预训练阶段从教师 BERT 的中间层学习知识。之后得到一个通用 TinyBERT，用作下游任务上特定任务蒸馏（TD）的学生模型初始化。

在初步实验中, 我们也尝试过用直接预训练得到的 BERTTINY 来初始化 TinyBERT, 再在下游任务上进行 TD。我们把这种压缩方法记为 BERTTINY(+TD)。表 7 的结果显示，BERTTINY(+TD) 在 MRPC 和 CoLA 任务上甚至比 BERTTINY 还差。我们推测，如果在预训练阶段不模仿 BERTBASE 的行为，BERTTINY 的中间表示（例如注意力矩阵和隐藏状态）分布会与 BERTBASE 模型不匹配。随后在微调过的 BERTBASE 监督下进行的特定任务蒸馏，会进一步扰乱 BERTTINY 已学到的分布/知识，最终导致在一些数据较少的任务上表现不佳。对于数据充足的任务（例如 MNLI），尽管预训练得到的分布已经被扰乱，TD 仍有足够的训练数据让 BERTTINY 很好地获得特定任务知识。

表 7： 预训练阶段不同方法的结果。TD 和 GD 分别指特定任务蒸馏（不含数据增强）和通用蒸馏。结果在开发集上评估。

系统	MNLI-m (392k)	MNLI-mm (392k)	MRPC (3.5k)	CoLA (8.5k)	平均
BERTTINY	75.9	76.9	83.2	19.5	63.9
BERTTINY(+TD)	79.2	79.7	82.9	12.4	63.6
TinyBERT(GD)	76.6	77.2	82.0	8.7	61.1
TinyBERT(GD+TD)	80.5	81.0	82.4	29.8	68.4

从表 7 的结果可以发现，即使不执行 MLM 和 NSP 任务，GD 也能有效地把知识从教师 BERT 迁移到学生 TinyBERT，并取得与 BERTTINY 相当的结果（61.1 对 63.9）。此外，通过继续从微调过的教师 BERTBASE 学习特定任务知识，特定任务蒸馏进一步提升了 TinyBERT 的表现。

> 译者注: MLM 指掩码语言模型（Masked Language Modeling），NSP 指下一句预测（Next Sentence Prediction），二者是 BERT 原始的预训练任务。

D GLUE 详情（GLUE Details）

GLUE 各数据集介绍如下:

MNLI。多体裁自然语言推理（Multi-Genre Natural Language Inference）是一个大规模、众包的蕴含（entailment）分类任务（Williams et al., 2018）。给定一对〈前提，假设〉（premise, hypothesis），目标是预测相对于前提，假设是蕴含、矛盾还是中立。

QQP。Quora 问题对（Quora Question Pairs）是来自 Quora 网站的问题对集合。任务是判断两个问题在语义上是否等价（Chen et al., 2018）。

QNLI。问题自然语言推理（Question Natural Language Inference）是斯坦福问答数据集（Stanford Question Answering Dataset）的一个版本，由 Wang et al.（2018）转换成二分类句对任务。给定一对〈问题，上下文〉，任务是判断上下文中是否包含问题的答案。

SST-2。斯坦福情感树库（Stanford Sentiment Treebank）是一个二分类单句任务，目标是预测电影评论的情感（Socher et al., 2013）。

CoLA。语言可接受性语料库（Corpus of Linguistic Acceptability）的任务是预测一个英文句子在语法上是否正确（Warstadt et al., 2019）。

STS-B。语义文本相似度基准（Semantic Textual Similarity Benchmark）是从新闻标题和许多其他领域抽取的句对集合（Cer et al., 2017）。任务是用 1 到 5 的分数评估两段文本有多相似。

MRPC。微软研究院释义语料库（Microsoft Research Paraphrase Corpus）是一个释义识别数据集，系统需要识别两个句子是否互为释义（Dolan and Brockett, 2005）。

RTE。识别文本蕴含（Recognizing Textual Entailment）是一个训练集较小的二分类蕴含任务（Bentivogli et al., 2009）。